

Programação Funcional em Haskell

Fundamentos funcionais

REALIZAR APÓS posição 16: 5.1 - Programação Funcional em Haskell: Testes de Propriedades - QuickCheck

Posições na playlist 1 a 16 | 120 minutos | 100 pontos

Curso principal: Programação Funcional em Haskell - Curso Completo - UFABC |

<https://www.youtube.com/watch?v=eTisiy5FB7k&list=PLYItvall0TqJ25sVTLcMhxsE0Hci58mpQ&index=1>

Conteúdo verificável: Paradigmas; lambda; combinador Y; tipos; where; guards; pattern matching; listas; recursão; QuickCheck

Instruções

Resolva sem consultar o gabarito. Apresente desenvolvimento, hipóteses, unidades e verificações. Código pode ser escrito em papel ou editor sem sugestões automáticas; use somente recursos compatíveis com o bloco. Ferramentas de simulação são permitidas apenas onde solicitadas. Use folhas adicionais para diagramas e cálculos.

Pontuação: Q1 = 15; Q2 = 25; Q3 = 25; Q4 = 35. Crédito parcial depende de raciocínio demonstrado. Meta: ≥ 80 prosseguir; 70–79 revisar erros; < 70 revisar o bloco.

Aluno(a): _____ Data: _____

Alinhamento: títulos/ordem e informações públicas do curso. As aulas não foram integralmente assistidas; detalhes não expostos não estão certificados.

Redução lambda

(a) Reduza $((\lambda x. \lambda y. x) a) b$. (b) Reduza $(\lambda x. x x)(\lambda z. z)$. (c) Explique substituição sem captura usando $(\lambda x. \lambda y. x) y$ e relacione combinador de ponto fixo à expressão recursiva sem exigir execução terminante.

Desenvolvimento / justificativa / esquema:

Guardas e recursão

Implemente em Haskell `somaPositivos :: [Int] -> Int`. (a) Use recursão e guardas ou pattern matching. (b) Trace `[-2, 3, 0, 5]`. (c) Dê caso base e explique pureza.

Desenvolvimento / justificativa / esquema:

Listas e compreensão

(a) Escreva compreensão que gera quadrados dos pares entre 1 e 10. (b) Escreva função recursiva equivalente sobre uma lista dada. (c) Explique diferença entre filtrar, transformar e ordenar.

Desenvolvimento / justificativa / esquema:

Propriedades e teste

Para tamanho recursivo de listas: (a) implemente; (b) formule propriedade QuickCheck $\text{tamanho}(xs++ys)=\text{tamanho } xs+\text{tamanho } ys$; (c) explique por que testes gerados não substituem prova e dê prova por indução em xs .

Desenvolvimento / justificativa / esquema:

Gabarito e Resoluções

Abra esta seção somente depois de concluir a tentativa. Há mais de um método correto possível; compare também o processo e os casos extremos.

Critérios de correção

Q1: 4 pontos pela modelagem/conceito, 8 pela solução, 3 pela verificação. Q2 e Q3: 6 pela modelagem, 14 pela solução, 5 pela justificativa/verificação. Q4: 8 pela modelagem, 20 pela solução, 7 pela análise crítica e casos extremos. Nos itens dissertativos, a solução inclui os conceitos e relações obrigatórios; crédito integral exige conectar esses conceitos ao cenário. Em código, valorize correção do contrato, casos base e explicação; uma chamada a biblioteca que evita a implementação pedida não substitui o algoritmo.

Questão 1 - Redução lambda (15 pontos)

(a) $\lambda y. a$ aplicado a b resulta a . (b) Substituir x por identidade dá $(\lambda z. z)(\lambda z. z)$, reduzindo a $\lambda z. z$. (c) Renomear variável ligada interna para $t: \lambda t. y$; resultado $\lambda y. y$ seria captura indevida. Um combinador de ponto fixo permite representar relação $Yf = f(Yf)$, mas essa igualdade não assegura terminação de toda avaliação. Indicar variáveis livres e ligadas é obrigatório.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 2 - Guardas e recursão (25 pontos)

```
somaPositivos [] = 0
somaPositivos (x:xs)
  | x > 0 = x + somaPositivos xs
  | otherwise = somaPositivos xs
```

Resultado 8. Lista diminui na chamada; vazia fornece identidade 0. A função não altera argumentos nem realiza efeitos de entrada/saída; o resultado depende dos valores de entrada. Aceitar compreensão com `sum` se explicar equivalência, mas solução recursiva é a referência pedida.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 3 - Listas e compreensão (25 pontos)

Compreensão `[x*x | x <- [1..10], even x]`, resultado `[4, 16, 36, 64, 100]`. Função: `vazia` → `[]`; se x é par, $x*x$: processa xs ; senão processa xs . Filtrar decide presença; transformar calcula cada resultado; nenhuma das duas ordena automaticamente. A sequência de entrada determina a ordem do resultado. Testar vazia, ímpares e negativos.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 4 - Propriedades e teste (35 pontos)

```
tamanho [] = 0
tamanho (_:xs) = 1 + tamanho xs
prop_concat xs ys = tamanho (xs ++ ys) == tamanho xs + tamanho ys
```

Restringir a assinatura da propriedade a listas de um tipo concreto ao executá-la, conforme ambiente do curso. `Base[]` é identidade; `passo(x:xs)` acrescenta 1 aos dois lados e usa hipótese. `QuickCheck` explora amostras, pode achar contraexemplo, mas não enumera necessariamente todo domínio. Confirmar que a propriedade testa a implementação relevante.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Diagnóstico do checkpoint

Registre sua nota e os conceitos a revisar em `progresso.csv`. Antes de avançar, refaça os itens errados sem olhar a resolução e crie um pequeno exemplo novo que use a mesma ideia. Nota alta sem justificar respostas não demonstra domínio.